

Проектирование микропроцессорной системы на базе МК i8051

Описание программной части

Листов 29

Инд. № подл.	Подп. и дата
Взам. инд. №	Инд. № дубл.
Подп. и дата	Подп. и дата

Выполнил: Казарин А.А.
Группа: А-12-23
Вариант: 8

Проверил: Михалин С.Н.

Москва, 2026

АННОТАЦИЯ

В данном документе приведено описание программы «Программа управления объектом», предназначенной для управления объектом микропроцессорной системой (МПС) на базе микроконтроллера AT89S53: ввода данных с клавиатуры и от внешнего устройства, формирования управляющих сигналов Y_1 и Y_2 и вывода информации на семисегментный индикатор.

Для функционирования данной программы необходима МПС, собранная по схеме электрической принципиальной варианта 8 (пояснительная записка по аппаратной части).

Исходным языком программы «Программа управления объектом» является ассемблер микроконтроллеров семейства MCS-51.

Основными функциями программы являются: инициализация МПС, сканирование клавиатуры, работа с кольцевым буфером на 1684 байт, выработка stroba Y_1 длительностью 271 мс, выработка сигнала Y_2 со стробом 2152 мкс, индикация на одном разряде.

Оформление программного документа «Описание программы» произведено по требованиям ЕСПД (ГОСТ 19.101-77, ГОСТ 19.103-77, ГОСТ 19.104-78, ГОСТ 19.105-78, ГОСТ 19.106-78, ГОСТ 19.402-78, ГОСТ 19.604-78).

СОДЕРЖАНИЕ

1. Общие сведения.....	4
2. Функциональное назначение.....	5
2.1. Классы решаемых задач.....	5
2.2. Назначение программы.....	5
2.3. Сведения о функциональных ограничениях на применение.....	5
3. Описание логической структуры.....	6
3.1. Алгоритм программы.....	6
3.2. Структура программы с описанием функций составных частей и связи между ними.....	18
3.3. Связи программы с другими программами.....	20
4. Входные данные.....	21
4.1. Клавиатура.....	21
4.2. Сигнал X2.....	21
5. Выходные данные.....	22
5.1. Характер и организация выходных данных.....	22
5.2. Формат, описание и способ кодирования выходных данных.....	22
ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ.....	24
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ.....	25
ПРИЛОЖЕНИЕ А. Листинги программ.....	26

1. ОБЩИЕ СВЕДЕНИЯ

Обозначение и наименование программы — «Программа управления объектом», вариант 8, группа А-12-23.

Программа низкого уровня не требует для своей работы иных программ. Исходным языком программирования является ассемблер MCS-51 (синтаксис Intel ASM51, совместимый с Keil A51 и MCU 8051 IDE).

Программы разработаны в интегрированной среде Keil μ Vision V5 (пакет PK51 для микроконтроллеров семейства 8051): трансляция — ассемблером A51, компоновка — BL51, отладка — встроенным симулятором среды.

2. ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ

2.1. Классы решаемых задач

- инициализация МПС;
- сканирование клавиатуры — ввод сигнала X_1 (число, введённое с клавиатуры);
- формирование сигнала Y_1 (строб заданной длительности);
- ввод сигнала X_2 (сохранение данных в кольцевом буфере);
- формирование сигнала Y_2 и сопровождающего строба;
- вывод символа на индикатор.

2.2. Назначение программы

МПС принимает информацию $\{X\}$ об объекте управления от датчиков, вырабатывает управляющие воздействия $\{Y\}$ в соответствии с заданным алгоритмом управления и подаёт их на исполнительные механизмы. Программа разбита на подзадачи по функциональному признаку:

- инициализация МПС;
- обработка прерываний: опрос клавиатуры, запись в буфер, таймеры генераторов сигналов Y ;
- определение кода нажатой клавиши;
- вывод данных на семисегментный индикатор;
- формирование сигнала Y_2 ;
- обеспечение работы кольцевого буфера (запись и чтение отсчётов X_2).

В соответствии с вариантом (табл. 1 ТЗ) разработан код трёх программ:

- программа 1 — обработчик прерывания клавиатуры (сканирование матрицы), код нажатой клавиши в аккумуляторе или 0FFh;
- программа 2 — чтение из кольцевого буфера (данные через аккумулятор) с управлением флагами пустоты и переполнения;
- программа 3 — расчёт и выдача сигнала Y_2 со стробом длительностью T_2 (прерывание таймера).

2.3. Сведения о функциональных ограничениях на применение

В качестве основы для построения МПС выбран микроконтроллер семейства MCS-51 — AT89S53, тактовая частота 12 МГц (машинный цикл 1 мкс). Таймер 2 и сторожевой таймер не используются (совместимость с i8051).

Интерфейсы:

- входные данные X_2 — 8-разрядное целое без знака по независимой параллельной шине со стробом записи низкого уровня в двухпортовое ОЗУ, нерегулярно, до 4 отсчётов в секунду;
- входные данные X_1 — десятичная цифра 0–9 с матричной клавиатуры 4×3 (сканирование по столбцам);
- выходной сигнал Y_2 — 8-разрядный параллельный код в выходном регистре со стробом низкого уровня длительностью 2152 мкс на выводе P1.2;
- выходной сигнал Y_1 — строб низкого уровня длительностью 271 мс на выводе P1.1;
- индикация — один семисегментный индикатор (общий анод), время индикации символа не более $T_3 = 2,740$ с.

3. ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ

3.1. Алгоритм программы

После сброса программа задаёт указатель стека, разрешает дешифратор адреса, устанавливает исходное состояние периферии и подготавливает таймеры и прерывания. Клавиатурный обработчик INT0 сканирует матрицу и возвращает код единственной нажатой клавиши; неоднозначное или пустое нажатие обозначается значением FFh. Готовность нового отсчёта внешнее устройство сообщает стробом на входе INT1; в обработчике микроконтроллер сам выбирает ячейку кольца и записывает в неё байт с разъёма через правый порт двухпортового буфера.

В основном цикле при ненулевом X1 и наличии данных считывается самый старый X2 и рассчитывается выход Y2. Для варианта 8 расчёт выполняется по формуле $(20 + X1 + X2) \bmod 256$; при $X1 = 0$ записывается нулевое значение. Значение сначала записывается во внешний регистр, затем активируется строб на время T2; завершение импульса обслуживает Timer1. Общий ход показан на рисунках 3.1 и 3.2, обработчики и подпрограммы — на рисунках 3.3–3.13.

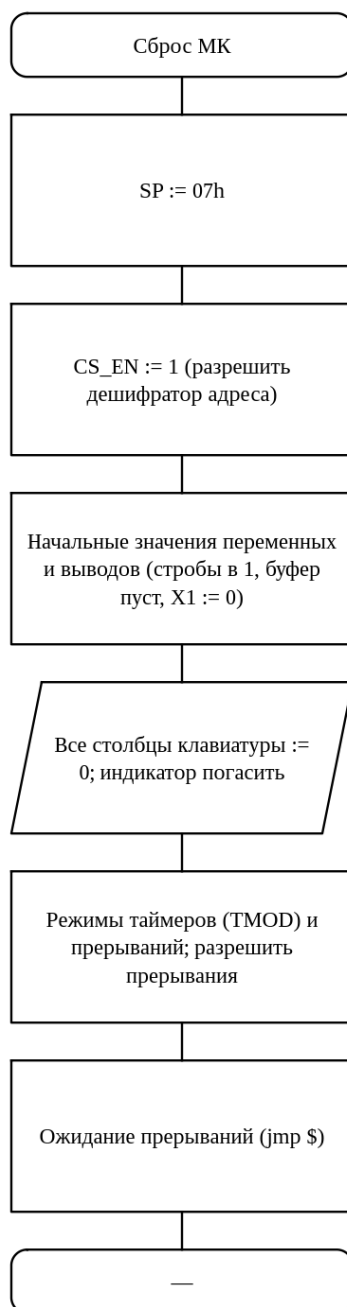


Рис. 3.1. Схема алгоритма: Инициализация МПС и основная программа

Сразу после сброса инициализация задаёт $SP = 07h$ и включает дешифратор, без чего обращения к внешним устройствам не выберут нужный узел. Затем задаются начальные значения указателей и флагов буфера, выходы стробов переводятся в пассивное состояние, а индикатор гасится. Все столбцы клавиатуры устанавливаются в ноль, чтобы нажатие могло сформировать $INT0$. После настройки режимов таймеров и прерываний микроконтроллер остаётся в ожидании событий.

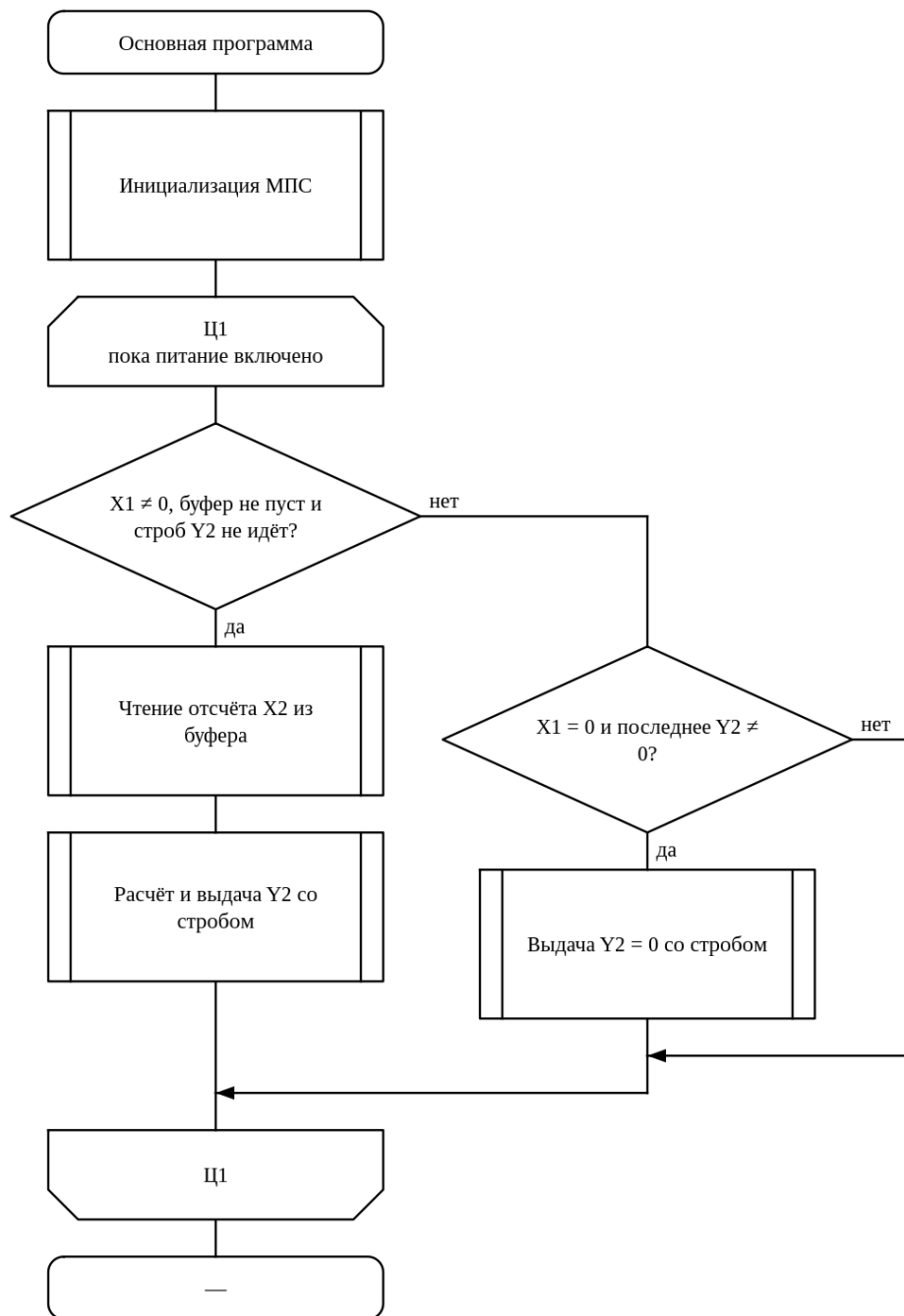


Рис. 3.2. Схема алгоритма: Основной цикл программы

Основной цикл проверяет, можно ли обработать очередной отсчёт: $X1$ должен быть ненулевым, в буфере должны быть данные, а предыдущий строб $Y2$ должен завершиться. Тогда из кольца читается $X2$ и вызывается расчёт $Y2$. При нулевом $X1$ система при необходимости один раз выдаёт пассивное значение, а затем возвращается к началу цикла. Сканирование клавиатуры и контроль длительности сигналов выполняются в обработчиках прерываний, поэтому цикл не ожидает окончания таймера.

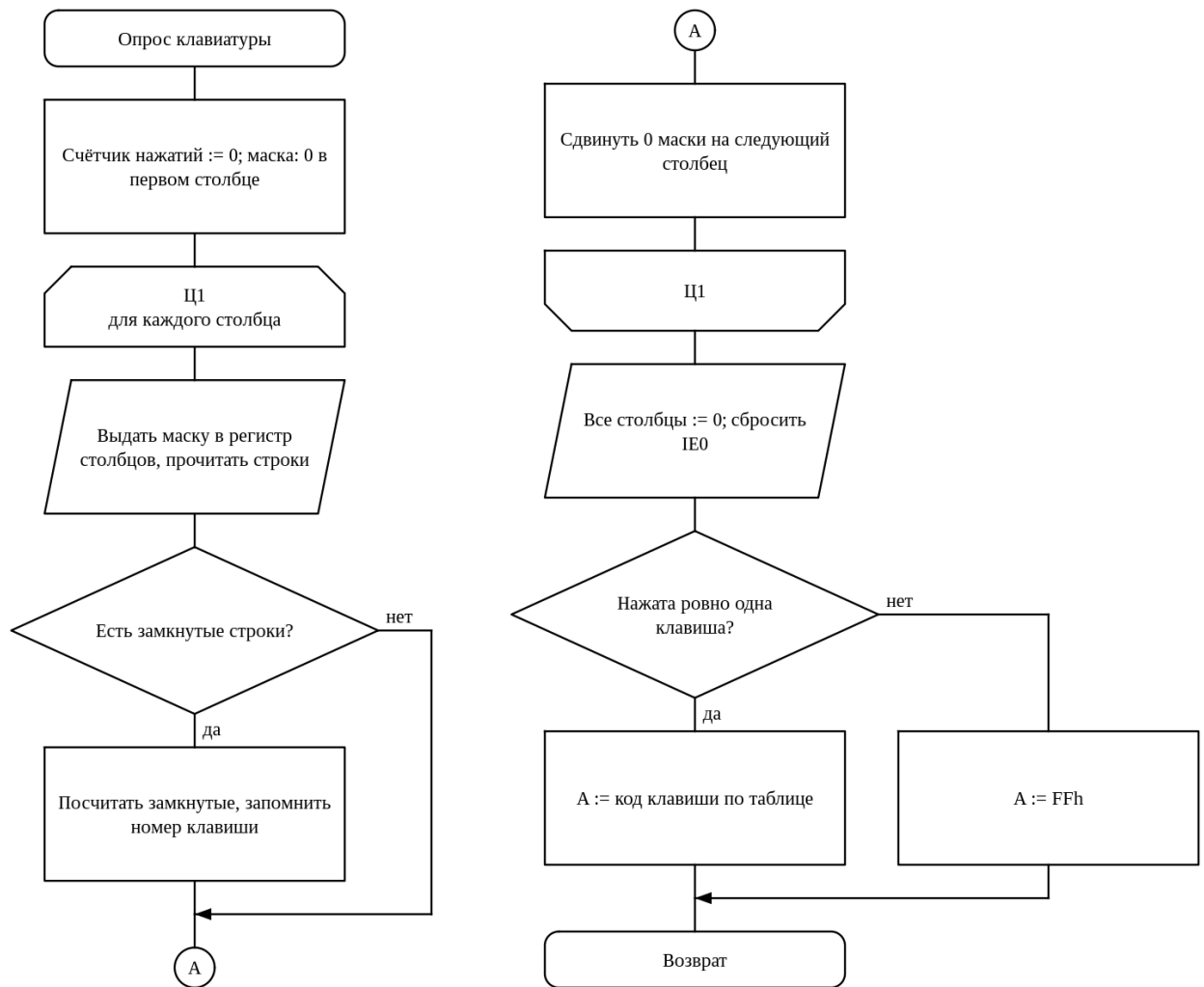


Рис. 3.3. Схема алгоритма: Опрос клавиатуры (обработчик INT0)

Опрос поочерёдно выбирает каждый столбец матрицы и читает состояние трёх строк. Старшие, неиспользуемые биты результата маскируются; низкий уровень строки означает замкнутую клавишу в выбранном столбце. Номер найденной позиции переводится в код по таблице раскладки варианта 8. В конце все столбцы возвращаются в ноль, а IE0 сбрасывается, чтобы переходы, возникшие во время сканирования, не вызвали повторный вход в обработчик.

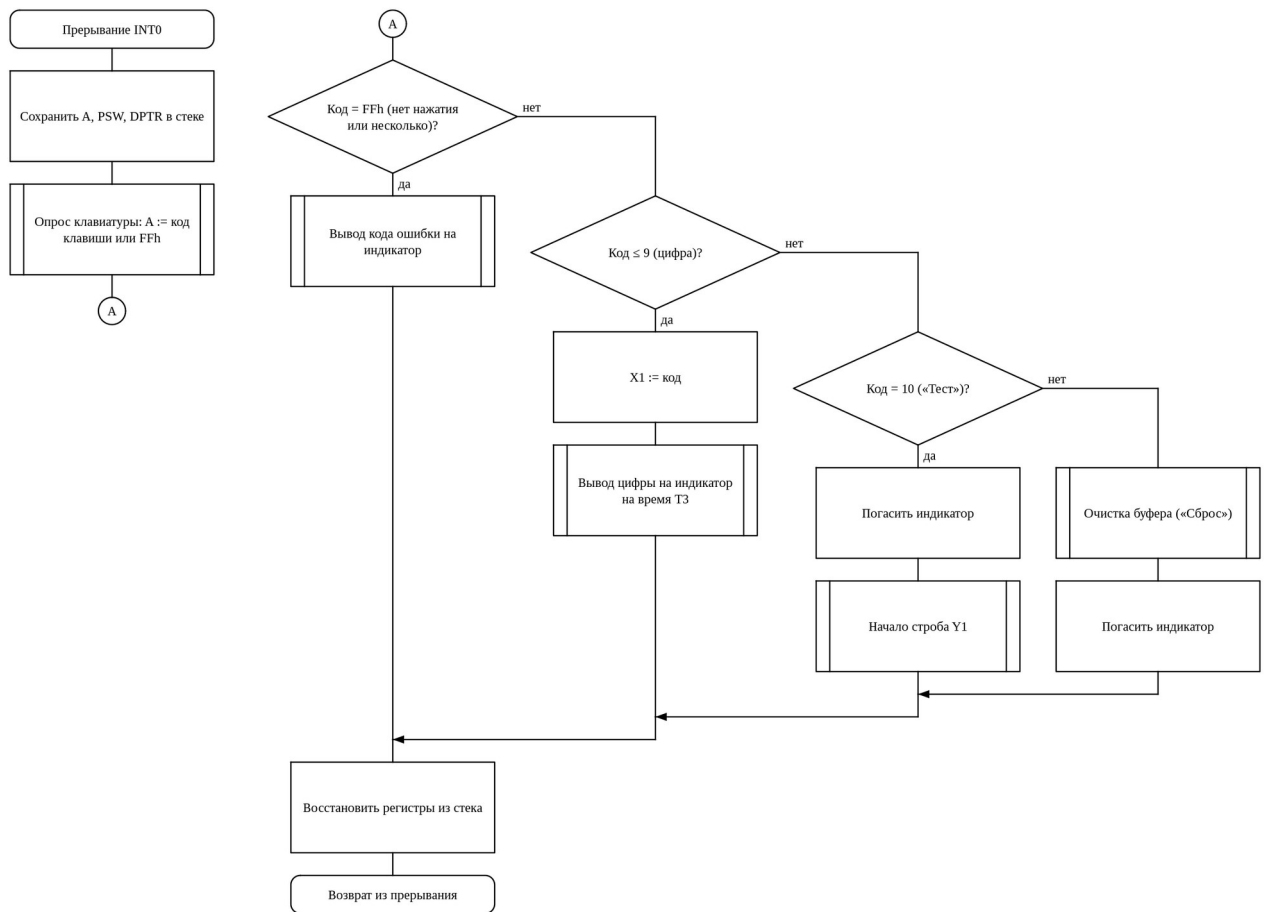


Рис. 3.4. Схема алгоритма: Обработчик прерывания INT0: реакция на клавишу

Обработчик INT0 сначала сохраняет регистры, которые использует опрос клавиатуры, и получает код нажатия. Если обнаружено ровно одно нажатие, код цифры можно передать основной логике; значение FFh означает отсутствие однозначного результата. Клавиши «Тест» и «Сброс» в полной системе запускают соответствующие действия с сигналом Y1 и буфером. Перед выходом сохранённые значения восстанавливаются, чтобы прерывание не повредило состояние прерванной программы.

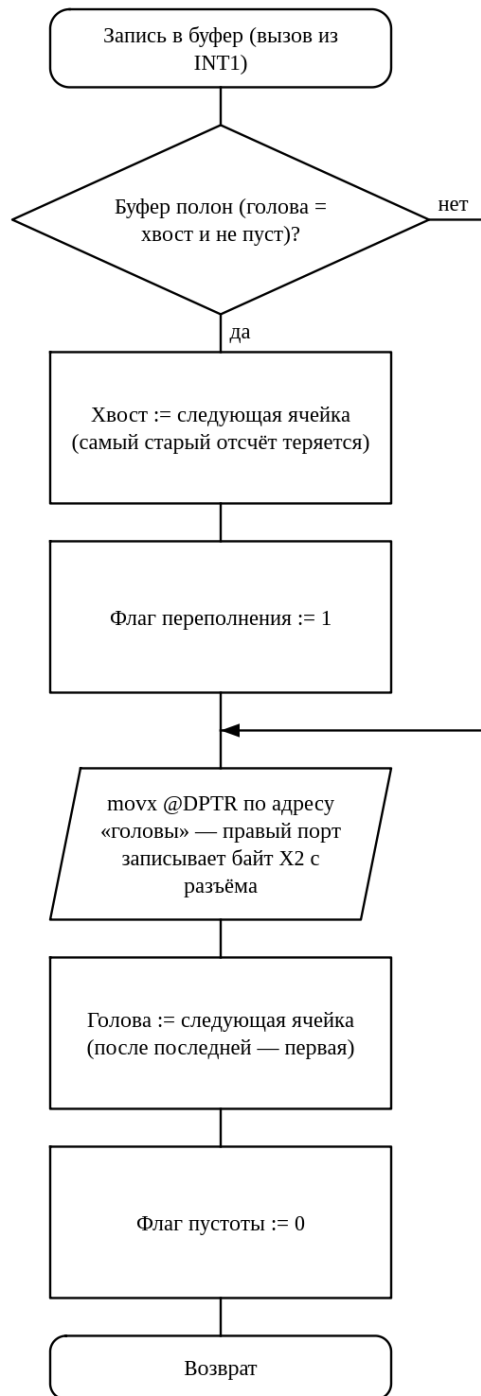


Рис. 3.5. Схема алгоритма: Запись отсчёта в кольцевой буфер

Подпрограмма записи вызывается из обработчика INT1 и обращается к ячейке, на которую указывает «голова»: команда `movx` по этому адресу заставляет правый порт буфера сохранить байт X2, который внешнее устройство держит на разъёме. После записи указатель сдвигается на следующую ячейку, а на конце выделенной области возвращается к BUF_START. Если свободного места нет, освобождается место за счёт самого старого значения и устанавливается флаг переполнения. Этот порядок сохраняет последовательность отсчётов и не позволяет записи выйти за пределы кольца.

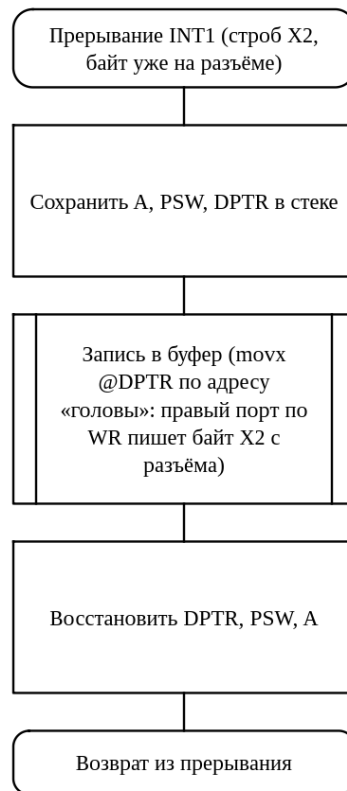


Рис. 3.6. Схема алгоритма: Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства

Перед стробом внешнее устройство выставляет очередной X2 на свой разъём, подключённый к данным правого порта IDT7005, а сам строб приходит на вход INT1. Обработчик сохраняет A, PSW и DPTR и вызывает запись в кольцо: в DPTR попадает адрес «головы», и команда `movx @DPTR, A` формирует этот адрес и импульс WR. Поскольку WR управляет записью только правого порта, в ячейку попадает байт с разъёма, а значение аккумулятора роли не играет; левый порт в это время закрыт, так как RD не активен. Затем регистры восстанавливаются и выполнение возвращается в прерванную программу.

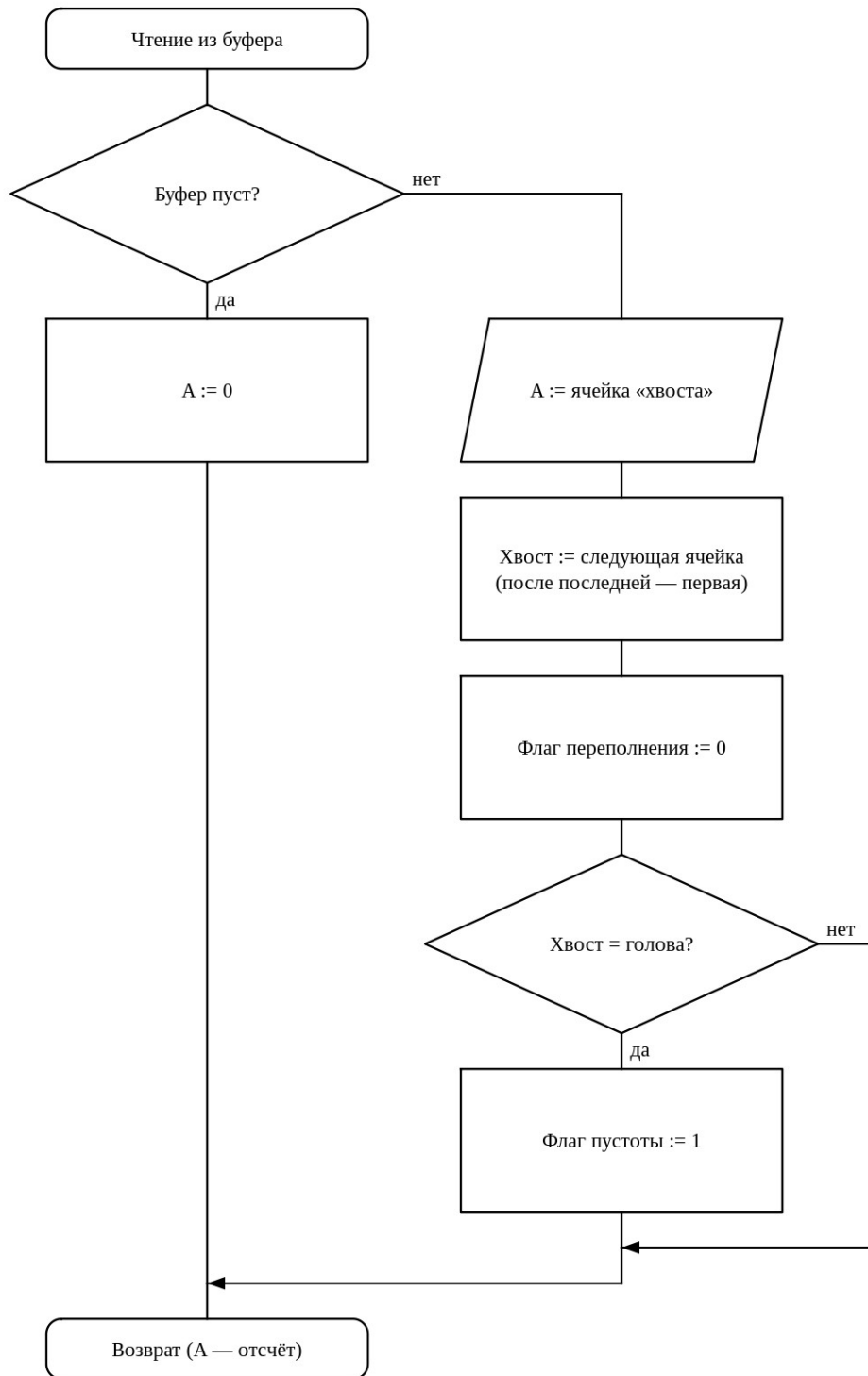


Рис. 3.7. Схема алгоритма: Чтение отсчёта из кольцевого буфера

Перед чтением проверяется флаг пустоты: если данных нет, подпрограмма возвращает ноль и оставляет указатели без изменений. Иначе значение считывается по адресу «хвоста», после чего хвост продвигается на одну ячейку. При переходе за конец буфера адрес возвращается к его началу. Если новый хвост совпал с головой, буфер помечается пустым; флаг переполнения очищается, потому что после чтения появилось свободное место.



Рис. 3.8. Схема алгоритма: Очистка кольцевого буфера (клавиша «Сброс»)

Очистка временно запрещает INT1, чтобы внешний запрос не обновил буфер между изменением указателей и флагов. Хвост приравняется к голове, поэтому очередь становится пустой без необходимости стирать сами ячейки памяти. Затем устанавливаются флаг пустоты и сбрасывается флаг переполнения. После этого INT1 снова разрешается.

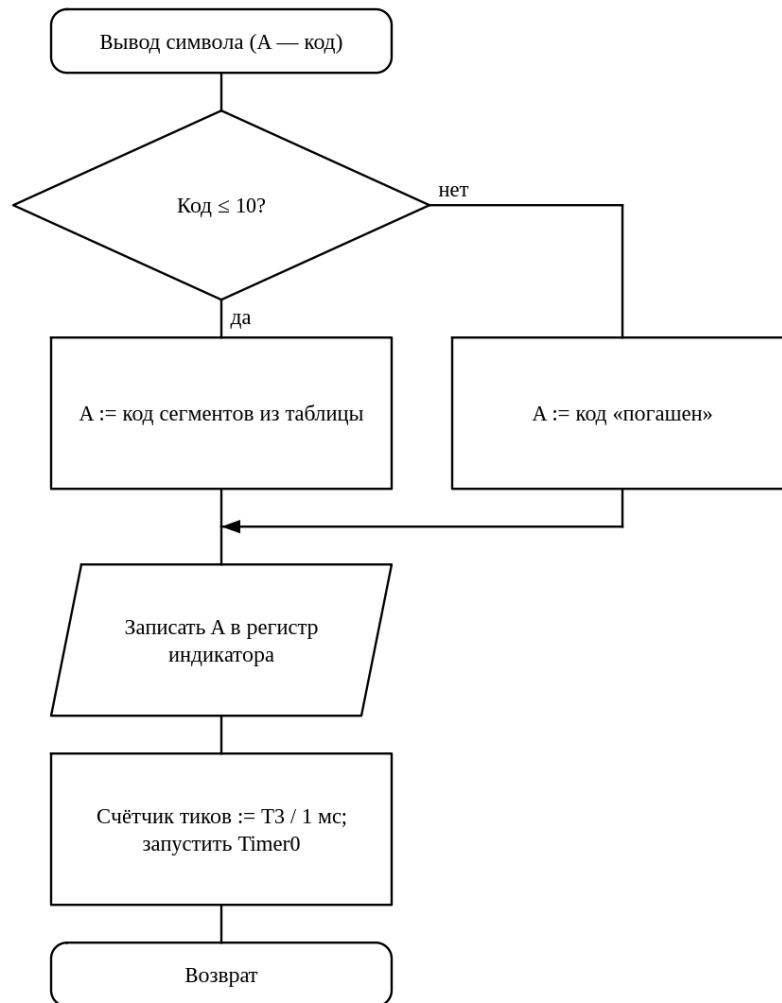


Рис. 3.9. Схема алгоритма: Вывод символа на индикатор

Для вывода сначала проверяется диапазон кода символа. Допустимое значение преобразуется в сегментный код из таблицы, а недопустимое заменяется состоянием «погашен». Полученный байт записывается в регистр индикатора по адресу `ADR_IND`. Таймер `T3` задаёт ограниченное время индикации, после которого обработчик таймера выключает сегменты.

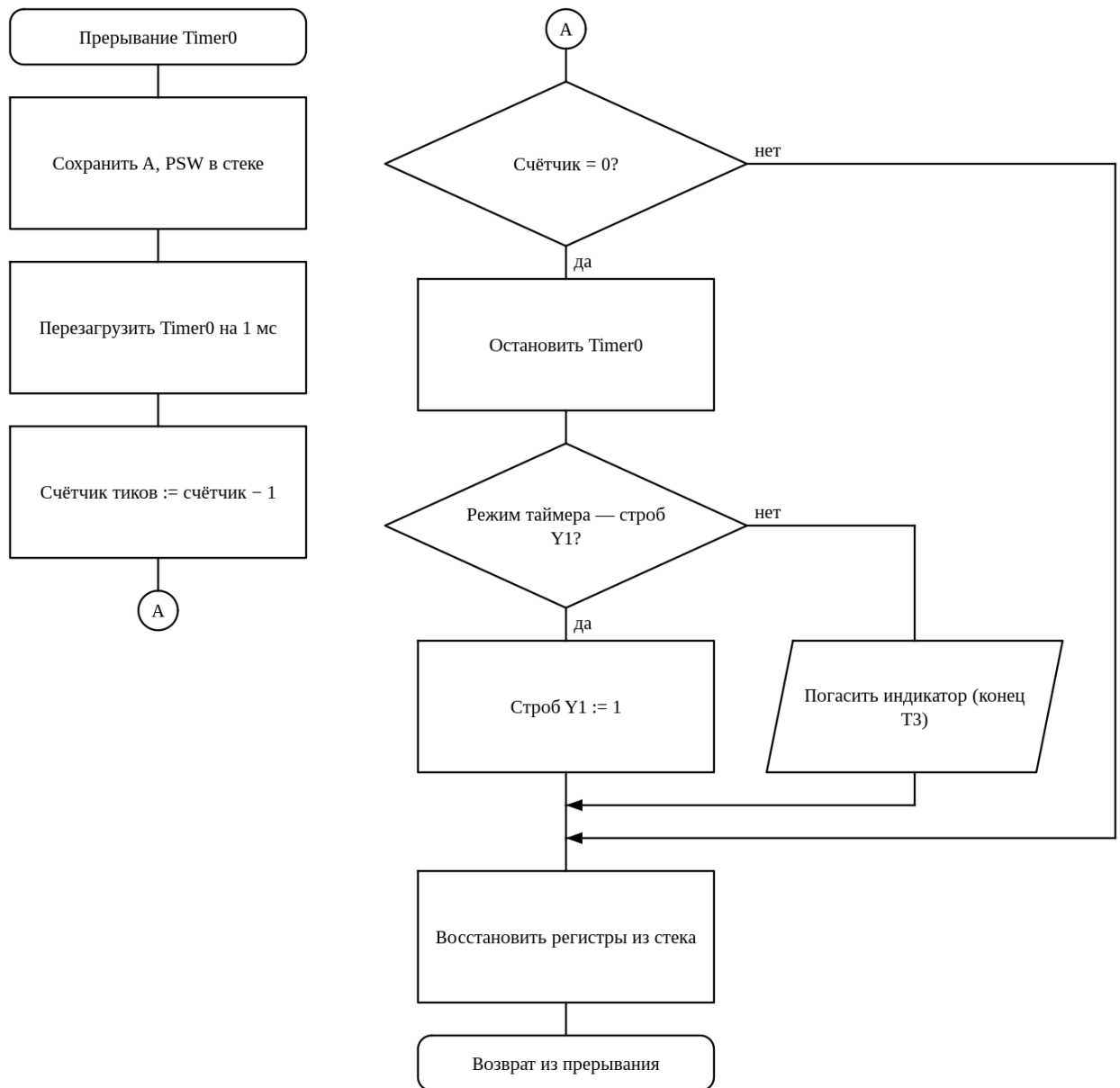


Рис. 3.10. Схема алгоритма: Обработчик прерывания Timer0: отсчёт T1 и T3

Каждое переполнение Timer0 начинает очередной миллисекундный интервал и уменьшает счётчик оставшихся тиков. Пока значение счётчика ненулевое, работа продолжается без ожидания в основном цикле. При достижении нуля таймер останавливается и выполняется действие, соответствующее его текущей задаче: завершается строб Y1 либо гасится индикатор после T3. Сохранение аккумулятора и PSW предотвращает порчу регистров прерванного кода.

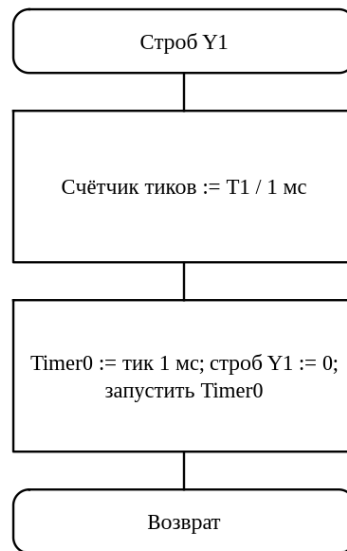


Рис. 3.11. Схема алгоритма: Начало строба Y1

Перед началом сигнала задаётся число миллисекундных тиков для длительности T1. Затем загружается начальное значение Timer0, и линия Y1 переводится в активный низкий уровень. Подпрограмма сразу возвращается, поэтому микроконтроллер может продолжать обработку клавиатуры и других событий. Timer0 завершит импульс и вернёт Y1 в единицу после заданного интервала.

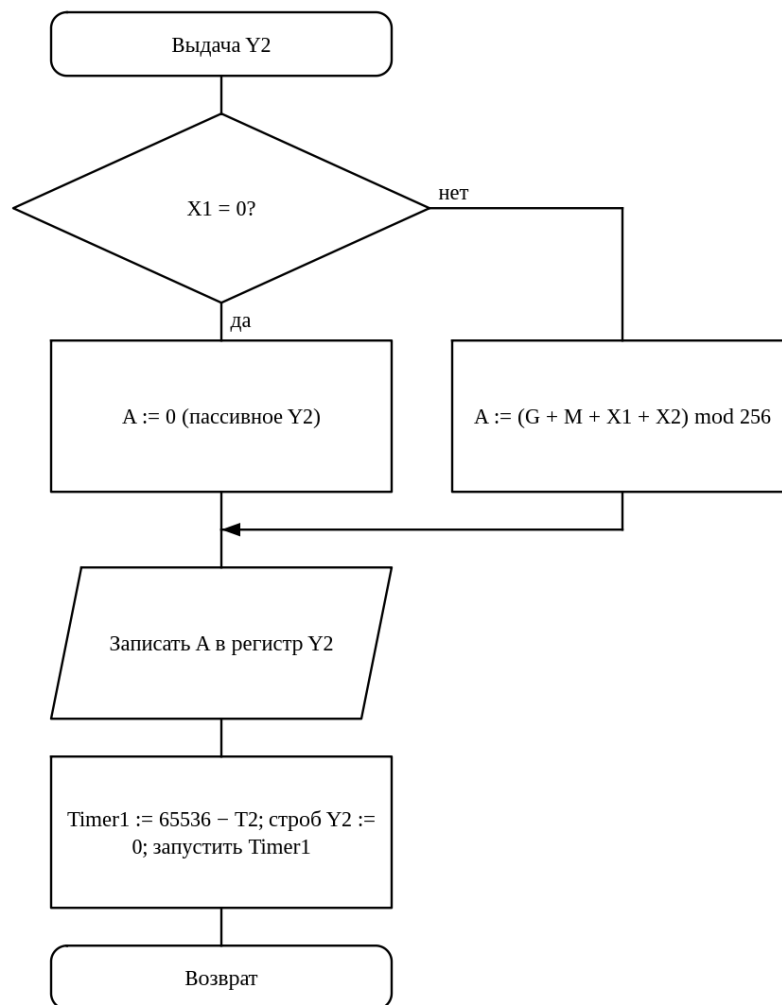


Рис. 3.12. Схема алгоритма: Расчёт и выдача сигнала Y2

Если X1 равен нулю, подпрограмма выбирает нулевой выход; в противном случае складывает X1, X2 и константу варианта Y2_CONST, отбрасывая перенос за пределы байта. Для варианта 8 эта константа равна 20. Сначала байт записывается в регистр Y2, затем линия P1.2 опускается и запускается Timer1 на T2 = 2152 мкс. Подпрограмма не ждёт окончания импульса: его закрывает отдельное прерывание таймера.

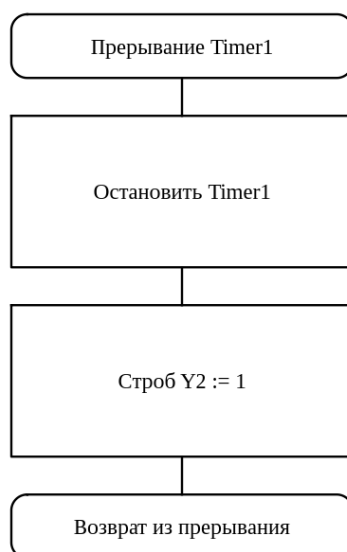


Рис. 3.13. Схема алгоритма: Обработчик прерывания Timer1: конец строба Y2

Когда Timer1 переполняется, заданная длительность строба истекла. Обработчик останавливает таймер и переводит PIN_Y2 в единицу, завершая активную часть сигнала. После этого выполнение возвращается к основному циклу, где может обрабатываться следующий отсчёт. Отдельный обработчик позволяет выдержать длительность без программного ожидания.

3.2. Структура программы с описанием функций составных частей и связи между ними

Программа состоит из инициализации, основной программы (ожидание прерываний) и подпрограмм, описанных ниже. Описание каждой подпрограммы взято из заголовочного комментария в исходном тексте (приложение А).

3.2.1 Подпрограмма KbScan (программа 1)

Назначение: сканирование матрицы клавиатуры по столбцам (обработчик INT0).

Входные данные: нет.

Выходные данные: A — код единственной нажатой клавиши по табл. 3 ТЗ или 0FFh.

Используемые регистры: A, DPTR, флаги PSW и временные переменные в DSEG.

3.2.2 Подпрограмма RecordKey (программа 1)

Назначение: учесть одну найденную замкнутую точку матрицы.

Входные данные: KCOL и KROWNO.

Выходные данные: KCOUNT увеличен; KCODE задан для первой клавиши.

Используемые регистры: A, DPTR, флаги PSW.

3.2.3 Подпрограмма BufRead (программа 2)

Назначение: чтение самого старого отсчёта из кольцевого буфера.

Входные данные: нет.

Выходные данные: A — отсчёт; F_EMPTY — буфер опустел; F_OVF сброшен (буфер больше не полон).

Используемые регистры: DPTR, флаги PSW; при пустом буфере возвращает A = 0.

3.2.4 Подпрограмма Y2Out (программа 3)

Назначение: расчёт $Y2 = (G+M+X1+X2) \bmod 256$ ($X1 = 0 \rightarrow Y2 = 0$), запись в регистр Y2 и строб T2.

Входные данные: X1, X2.

Выходные данные: регистр Y2; PIN_Y2 = 0 на T2 мкс (конец — в прерывании Timer1).

Используемые регистры: A, DPTR, флаги PSW, Timer1.

3.2.5 Обработчик прерывания Timer1 (программа 3)

Назначение: Timer1 — конец строба Y2.

Распределение ресурсов микроконтроллера — прерывания, адреса внешних устройств и переменные — приведено в таблицах 3.1–3.3.

Таблица 3.1

Прерывания

Источник	Вектор	Назначение	Запуск
INT0 (P3.2)	0003h	нажатие клавиши	по спаду
Timer0	000Bh	отсчёт T_1 (271 мс) и T_3 (2740 мс), тик 1 мс	переполнение
INT1 (P3.3)	0013h	строб записи X_2	по спаду
Timer1	001Bh	отсчёт T_2 (2152 мкс)	переполнение
UART	0023h	не используется (заглушка)	—

Таблица 3.2

Адреса внешних устройств (обращение командами movx при P3.4 = 1)

Устройство	Выход дешифратора	Адреса
Индикатор	Y7	0E000h–0FFFFh
Клавиатура	Y1	2000h–3FFFh
Буфер (IDT7005)	Y5	0A000h–0BFFFh
Регистр Y2	Y6	0C000h–0DFFFh

Таблица 3.3

Переменные во внутреннем ОЗУ

Имя	Адрес	Назначение
HEAD_L, HEAD_H	47h, 48h	«голова» — адрес ячейки буфера для следующей записи
TAIL_L, TAIL_H	49h, 4Ah	«хвост» — адрес ячейки буфера для следующего чтения
F_EMPTY	21h.0 (бит 08h)	флаг пустоты буфера
F_OVF	22h.4 (бит 14h)	флаг переполнения буфера
KCOL	30h	переменная программы 1
TEMP	30h	переменная программы 2
X1	30h	сигнал X_1 — цифра с клавиатуры
KROWS	31h	переменная программы 1
X2	31h	сигнал X_2 — отсчёт из буфера
KROWNO	32h	переменная программы 1
KCOUNT	33h	переменная программы 1
KCODE	34h	переменная программы 1

3.3. Связи программы с другими программами

Программа «Программа управления объектом» в ходе своей работы не использует других программ.

4. ВХОДНЫЕ ДАННЫЕ

4.1. Клавиатура

Клавиатура — матрица 4×3 (4 столбцов × 3 строк), сканирование по столбцам: в регистр столбцов (адрес 2000h, разряды D0...D3) выдаётся 0 в опрашиваемый столбец, строки читаются через повторитель по тому же адресу (разряды D0...D2, 0 — контакт замкнут). Нажатие любой клавиши при всех столбцах в 0 вызывает прерывание INT0. Коды клавиш (табл. 3 ТЗ) и их расположение — в таблице 4.1.

Таблица 4.1

Расположение клавиш и их коды («клавиша» — код)

	Col1	Col2	Col3	Col4
Row1	«1» — 1	«2» — 2	«3» — 3	«Т» — 10
Row2	«4» — 4	«5» — 5	«6» — 6	«С» — 11
Row3	«7» — 7	«8» — 8	«9» — 9	«0» — 0

Результат опроса — код единственной нажатой клавиши (0–9 — цифры, 10 — «Тест», 11 — «Сброс») или 0FFh, если не нажато ничего или нажато больше одной клавиши. Цифра становится значением X_1 (начальное значение $X_1 = 0$).

4.2. Сигнал X2

X_2 — 8-разрядное целое без знака. Внешнее устройство записывает отсчёт в двухпортовое ОЗУ IDT7005 через свой порт и сопровождает запись стробом, который вызывает прерывание INT1. Отсчёты хранятся в кольцевом буфере на 1684 байт (адреса 0A000h–0A693h).

5. ВЫХОДНЫЕ ДАННЫЕ

5.1. Характер и организация выходных данных

Индикация. На индикаторе отображается последняя введенная цифра — до ввода следующей, но не дольше $T_3 = 2,740$ с; при ошибке — код ошибки с горящей точкой др. Начальное состояние — индикатор погашен.

Кольцевой буфер во внешнем ОЗУ. Ячейки 0A000h–0A693h. «Голова» — адрес ячейки для следующей записи, «хвост» — для следующего чтения; после последней ячейки следует первая. Буфер пуст — флаг пустоты = 1 (голова = хвост). Если буфер заполнен, новый отсчет затирает самый старый (хвост сдвигается) и устанавливается флаг переполнения; после чтения флаг переполнения сбрасывается.

Управляющие сигналы. Y_1 — строб низкого уровня длительностью 271 мс по клавише «Тест». $Y_2 = (12 + 8 + X_1 + X_2) \bmod 256$ при $X_1 \neq 0$, иначе $Y_2 = 0$ (пассивное состояние); вырабатывается, пока в буфере есть данные и $X_1 \neq 0$. Каждая запись Y_2 в выходной регистр сопровождается стробом длительностью 2152 мкс; до записи регистр выдает предыдущее значение.

Расчётные параметры работы программы. Константы таймеров (кварц 12 МГц, машинный цикл $T_{мц} = 12 / f = 1$ мкс, режим 1 — 16 бит):

ТН1:ТЛ1 = 65536 – $T_2 / T_{мц}$ = 65536 – 2152 = 63384 = 0F798h (строб Y_2 , таймер 1)

ТН0:ТЛ0 = 65536 – 1 мс / $T_{мц}$ = 65536 – 1000 = 64536 = 0FC18h (тик 1 мс, таймер 0)

$N_{T1} = T_1 / 1 \text{ мс} = 271$; $N_{T3} = T_3 / 1 \text{ мс} = 2740$ (число тиков)

Минимальное время заполнения буфера (при $X_1 = 0$ сигнал Y_2 пассивен и буфер не читается, отсчеты поступают с максимальной интенсивностью):

$t_{\min} = V / f_{X2} = 1684 / 4 = 421,0$ с (7,0 мин).

Максимальная глубина стека для программы 1 — 9 байт (SP от 07h до 10h). Значение получено прогоном программы в симуляторе на всех сценариях проверки (каждая клавиша, одновременные нажатия, помеха на INT0) и складывается из адреса возврата при входе в обработчик INT0 (2 байта), адреса возврата при вызове процедуры опроса (2 байта) и сохраняемых в стеке регистров.

5.2. Формат, описание и способ кодирования выходных данных

Код символа (табл. 2 ТЗ) преобразуется в код сегментов по таблице 5.1: разряды D0...D7 регистра индикатора соответствуют сегментам a, b, c, d, e, f, g, dr; индикатор с общим анодом — сегмент горит при 0.

Таблица 5.1

Коды сегментов индикатора (dp g f e d c b a)			
Код	Символ	Двоичный	Шестн.
0	«0»	11000000	0C0h
1	«1»	11111001	0F9h
2	«2»	10100100	0A4h
3	«3»	10110000	0B0h
4	«4»	10011001	99h
5	«5»	10010010	92h
6	«6»	10000010	82h
7	«7»	11111000	0F8h
8	«8»	10000000	80h
9	«9»	10010000	90h
10	«E»	10000110	86h
11 и больше	погашен	11111111	0FFh

Таблица 5.2

Коды ошибок (символ с горящей точкой др, индикация до нажатия любой клавиши, но не дольше ТЗ)

Индикация	Смысл	Когда выставляется
«1.»	нажато несколько клавиш	опрос клавиатуры нашёл больше одного замкнутого контакта
«2.»	нажатие не распознано	прерывание пришло, а замкнутых контактов нет
«3.»	переполнение буфера	запись X_2 затёрла непрочитанный отсчёт
«4.»	повторный «Тест»	«Тест» нажат, пока идёт строб Y_1

Обработчик клавиатуры возвращает FFh, если нажатий нет или замкнулось несколько клавиш; вызывающий код должен трактовать это как нераспознанный ввод. Программы варианта 8 реализуют отдельные задания по сканированию, чтению буфера и формированию Y_2 ; вывод кодов ошибок на индикатор относится к алгоритму полной системы и в этих трёх отдельных исходных файлах не реализован.

Сигнал Y_2 выдаётся 8-разрядным кодом в выходной регистр (адрес 0C000h), строб — на выводе P1.2; сигнал Y_1 — на выводе P1.1.

В 16-разрядном режиме (режим 1) таймер 0 перезагружается в обработчике прерывания; задержка входа в прерывание (несколько машинных циклов на тик) компенсируется поправкой к загружаемому значению или учитывается как погрешность длительности. В режиме 2 (8 разрядов с автоперезагрузкой) перезагрузку выполняет сам таймер, а тик набирается программным счётчиком переполнений.

ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ

Таблица 5.3

Сокращения

Сокращение	Расшифровка
МК	микроконтроллер
МПС	микропроцессорная система
ОЗУ	оперативное запоминающее устройство
ПЗУ	постоянное запоминающее устройство
ЕСПД	единая система программной документации
ТЗ	техническое задание
CS	Chip Select, выбор микросхемы
IRQ	запрос прерывания
SP	указатель стека
DPTR	16-разрядный регистр-указатель данных
dp	десятичная точка индикатора

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]

ПРИЛОЖЕНИЕ А

Листинги программ

Программа 1 — файл «Казарин АА-код-1»

```
; Казарин А.А., А-12-23, 8, v1
$INCLUDE (vars.inc) ; адреса, биты, константы варианта; в файл для робота mpscode вклеит его текст
DSEG AT 30h
KCOL: DS 1 ; текущий столбец 0..3
KROWS: DS 1 ; маска нажатых строк
KROWNO: DS 1 ; номер строки для таблицы кодов
KCOUNT: DS 1 ; число найденных клавиш
KCODE: DS 1 ; код единственной клавиши
CSEG AT 0h
sjmp START
org 03h ; INT0 — нажатие клавиши: процедура вызывается отсюда
call KbScan ; %proc%
reti
org 0bh ; "заглушка" для Tmr0_ovf
nop
reti
org 13h ; "заглушка" для INT1
nop
reti
org 1bh ; "заглушка" для Tmr1_ovf
nop
reti
org 23h ; "заглушка" для UART
nop
reti
org 2bh
START:
; Инициализация МК *****
mov SP, #07h
setb PIN_CSEN ; включить дешифратор адресов
mov DPTR, #ADR_KB ; подготовить регистр столбцов
clr A
movx @DPTR, A ; все столбцы активны для INT0
setb IT0 ; прерывание INT0 по спаду
setb EX0 ; разрешить клавиатурное прерывание
setb EA ; разрешить глобальные прерывания
jmp $ ; %stop%

; KbScan — сканирование матрицы клавиатуры по столбцам (обработчик INT0).
; Вход: нет. Выход: A — код единственной нажатой клавиши по табл. 3 ТЗ или 0FFh.
; Портит: A, DPTR, флаги PSW и временные переменные в DSEG.
KbScan:
push PSW ; сохранить рабочие флаги и адрес данных
push DPL
push DPH
mov KCOUNT, #0 ; начать подсчёт нажатий
mov KCODE, #0FFh ; значение на случай неоднозначного ввода
mov KCOL, #0 ; начать с первого столбца
KbColumn:
mov A, KCOL
mov DPTR, #ColMask ; выбрать маску столбца
movc A, @A+DPTR
mov DPTR, #ADR_KB
movx @DPTR, A ; опустить только выбранный столбец
movx A, @DPTR ; прочитать строки матрицы
anl A, #((1 SHL KB_ROWS) - 1)
cpl A
anl A, #((1 SHL KB_ROWS) - 1)
mov KROWS, A

mov A, KROWS
jnb ACC.0, KRow2 ; строка 1 замкнулась на столбец?
mov KROWNO, #0
call RecordKey
KRow2:
mov A, KROWS
jnb ACC.1, KRow3 ; строка 2 замкнулась на столбец?
mov KROWNO, #1
call RecordKey
KRow3:
mov A, KROWS
jnb ACC.2, KNextColumn ; строка 3 замкнулась на столбец?
mov KROWNO, #2
call RecordKey
KNextColumn:
inc KCOL ; перейти к следующему столбцу
mov A, KCOL
```

```

cjne A, #KB_COLS, KbColumn

mov DPTR, #ADR_KB
clr A
movx @DPTR, A          ; вернуть матрицу в режим ожидания
clr IE0                ; убрать спады INT0 во время сканирования
mov A, KCOUNT
cjne A, #1, KbInvalid  ; принимать только одно нажатие
mov A, KCODE
pop DPH                ; восстановить регистры, не меняя код в A
pop DPL
pop PSW
ret
KbInvalid:
mov A, #0FFh
pop DPH                ; восстановить регистры, не меняя код в A
pop DPL
pop PSW
ret

; RecordKey — учесть одну найденную замкнутую точку матрицы.
; Вход: KCOL и KROWNO. Выход: KCOUNT увеличен; KCODE задан для первой клавиши.
; Портит: A, DPTR, флаги PSW.
RecordKey:
inc KCOUNT            ; каждое замыкание — отдельное нажатие
mov A, KCOUNT
cjne A, #1, RecordDone ; код сохраняется только для первой клавиши
mov A, KCOL
rl A
add A, KCOL            ; индекс столбца умножен на число строк
add A, KROWNO
mov DPTR, #KeyMap
movc A, @A+DPTR        ; получить код клавиши по таблице раскладки
mov KCODE, A
RecordDone:
ret

; ColMask: при опросе один столбец имеет ноль, остальные — единицы.
ColMask:
db 0FEh, 0FDh, 0FBh, 0F7h
; KeyMap расположена столбцами: 1/4/7, 2/5/8, 3/6/9, T/C/0.
KeyMap:
db 1, 4, 7, 2, 5, 8, 3, 6, 9, 10, 11, 0
END

```

Программа 2 — файл «Казарин АА-код-2»

```

; Казарин А.А., А-12-23, 8, v1
$INCLUDE (vars.inc)          ; адреса, биты, константы варианта; в файл для работа mpcode вклеит его текст
DSEG AT 30h
TEMP: DS 1
CSEG AT 0h
sjmp START
org 03h ; "заглушка" для INT0
nop
reti
org 0bh ; "заглушка" для Tmr0_ovf
nop
reti
org 13h ; "заглушка" для INT1
nop
reti
org 1bh ; "заглушка" для Tmr1_ovf
nop
reti
org 23h ; "заглушка" для UART
nop
reti
org 2bh
START:
; Инициализация МК *****
mov SP, #07h
setb PIN_CSEN             ; разрешить выбор внешней памяти
mov HEAD_L, #LOW BUF_START ; начать запись с первой ячейки
mov HEAD_H, #HIGH BUF_START
mov TAIL_L, #LOW BUF_START ; начать чтение с той же ячейки
mov TAIL_H, #HIGH BUF_START
setb F_EMPTY              ; исходный буфер пуст
clr F_OVF                 ; переполнения ещё не было
setb EX1                  ; разрешить внешний запрос X2
setb EA                   ; разрешить прерывания глобально
call BufRead ; %proc%
jmp $ ; %stop%

; BufRead — чтение самого старого отсчёта из кольцевого буфера.

```

```

; Вход: нет. Выход: A — отсчёт; F_EMPTY — буфер опустел; F_OVF сброшен (буфер больше не полон).
; Портит: DPTR, флаги PSW; при пустом буфере возвращает A = 0.
BufRead:
    clr EX1                                ; не менять указатели во время чтения
    jb F_EMPTY, BufEmpty
    mov DPL, TAIL_L                        ; указать на старейший отсчёт
    mov DPH, TAIL_H
    movx A, @DPTR                          ; получить значение из XRAM
    mov TEMP, A                            ; сохранить результат при обновлении указателя
    mov A, TAIL_L                          ; увеличить младшую часть адреса
    add A, #1
    mov TAIL_L, A
    mov A, TAIL_H                          ; перенести переполнение в старшую часть
    addc A, #0
    mov TAIL_H, A
    cjne A, #HIGH BUF_END, BufCompareHead
    mov A, TAIL_L
    cjne A, #LOW BUF_END, BufCompareHead
    mov TAIL_L, #LOW BUF_START ; вернуться к началу кольца
    mov TAIL_H, #HIGH BUF_START
BufCompareHead:
    mov A, TAIL_L                          ; последний элемент прочитан?
    cjne A, HEAD_L, BufStillHasData
    mov A, TAIL_H
    cjne A, HEAD_H, BufStillHasData
    setb F_EMPTY                          ; голова догнана хвостом: данных нет
BufStillHasData:
    clr F_OVF                             ; после чтения место снова свободно
    mov A, TEMP                            ; вернуть отсчёт вызывающей программе
    setb EX1                               ; разрешить новые X2
    ret
BufEmpty:
    mov A, #0                             ; пустой буфер возвращает ноль
    setb EX1                               ; восстановить разрешение INT1
    ret
END

```

Программа 3 — файл «Казарин АА-код-3»

```

; Казарин А.А., А-12-23, 8, v1
$INCLUDE (vars.inc)                    ; адреса, биты, константы варианта; в файл для работа mpcode вклеит его текст
DSEG AT 30h
X1:   DS 1                             ; цифра с клавиатуры (0 — Y2 пассивен)
X2:   DS 1                             ; отсчёт из буфера
CSEG AT 0h
    sjmp START
org 03h ; "заглушка" для INT0
    nop
    reti
org 0bh ; "заглушка" для Tmr0_ovf
    nop
    reti
org 13h ; "заглушка" для INT1
    nop
    reti
org 1bh ; Timer1 — конец строба Y2
    clr TR1                             ; остановить отсчёт длительности
    setb PIN_Y2                         ; вернуть строб в пассивное состояние
    reti
org 23h ; "заглушка" для UART
    nop
    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h
    setb PIN_CSEN                       ; включить дешифратор адреса
    setb PIN_Y2                         ; строб неактивен до первого вызова
    mov TMOD, #11h                     ; таймеры в режиме 1
    clr TR1                             ; Timer1 остановлен
    setb ET1                           ; разрешить прерывание Timer1
    setb EA                             ; разрешить прерывания глобально
    mov X1, #0                         ; пример: нулевой ввод держит Y2 пассивным
    mov X2, #0
    call Y2Out ; %proc%
    jmp $ ; %stop%

; Y2Out — расчёт Y2 = (G+M+X1+X2) mod 256 (X1 = 0 → Y2 = 0), запись в регистр Y2 и строб T2.
; Вход: X1, X2. Выход: регистр Y2; PIN_Y2 = 0 на T2 мкс (конец — в прерывании Timer1).
; Портит: A, DPTR, флаги PSW, Timer1.
Y2Out:
    mov A, X1                          ; нулевой X1 требует нулевого выхода
    jz Y2Zero
    add A, X2                          ; сложение выполняется по модулю 256
    add A, #Y2_CONST                   ; прибавить G + M из vars.inc

```

```
        sjmp Y2Write
Y2Zero:  clr A                                ; пассивное значение при X1 = 0
Y2Write: mov DPTR, #ADR_Y2                  ; выбрать внешний регистр Y2
        movx @DPTR, A                      ; записать значение до начала стробы
        mov TH1, #T2_RELOAD_H             ; загрузить длительность T2
        mov TL1, #T2_RELOAD_L
        clr TF1                           ; сбросить старое переполнение
        clr PIN_Y2                        ; активная часть стробы начинается
        setb TR1                          ; Timer1 завершит строб по прерыванию
        ret
END
```